TECHNICAL RESEARCH & SYSTEM DOCUMENTATION

# Intent-Aware Deception System for Real-Time Attacker Profiling

Complete Technical Architecture, Frontend Operations & Empirical ML Evaluation Reference

| | |
|---|---|
| **System Version:** 1.2.0 (Production Verified) | **Software Tests:** 166 Passed / 0 Failed |
| **Primary Held-Out Accuracy:** 100.0% (Stratified) | **Robustness (LFO):** 81.98% (Macro-F1 0.8153) |
| **ML Model Architecture:** Calibrated LinearSVC | **Mean Latency:** 11.83 ms per command |
| **Dataset Corpus:** 6,867 Unique Commands (243 Families) | **Honeypot Sensor:** Cowrie SSH/Telnet Container |

**Figure 4.1: INT-AWARE System Architecture and Data Pipeline**

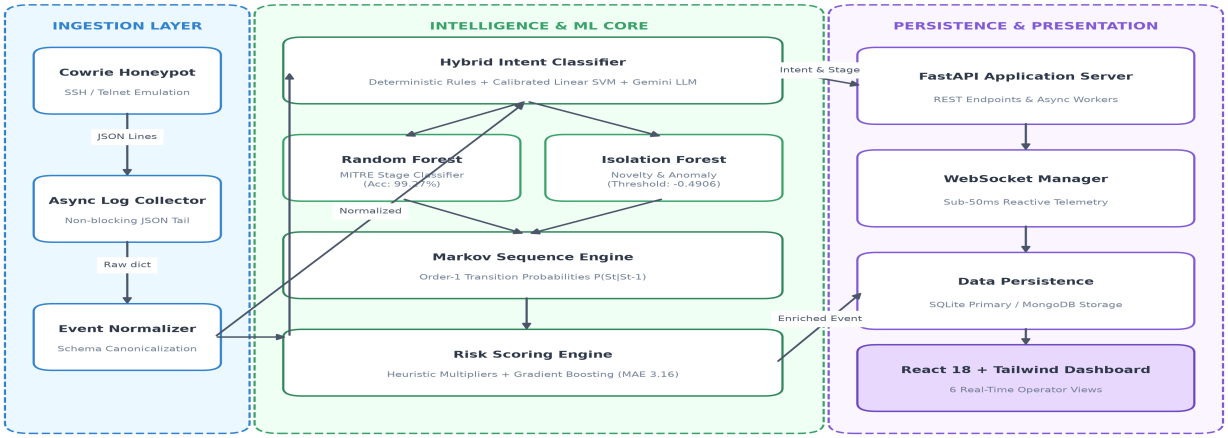



*Figure 1: End-to-end Deception System Topology & Telemetry Processing Architecture*

# 1. Project Overview & Operational Objectives

**1.1 Problem Statement:** Modern cyber adversaries rapidly bypass perimeter defenses using compromised credentials or novel exploitation techniques. Once inside a network, their actions manifest as shell command sequences across Linux hosts. Conventional intrusion detection systems (IDS) either generate high false alarm rates or rely on brittle static signatures. Traditional honeypots either pose significant security hazards by hosting live vulnerable kernels or lack dynamic behavioral realism.

**1.2 System Purpose:** The *Intent-Aware Deception System for Real-Time Attacker Profiling* solves this dilemma by coupling an isolated, containerized Cowrie honeypot sensor with a real-time machine learning intent classifier and a dynamic deception engine. It observes incoming attacker commands, strips syntactic noise, maps the underlying intent to one of 10 canonical cyber intrusion categories, predicts multi-step attack progression via first-order Markov chains, and dynamically serves synthetic deceptive artifacts—all without executing attacker payloads on the host operating system.

**1.3 Key Beneficiaries:** Enterprise Security Operations Centers (SOCs), threat intelligence analysts, incident response teams, and cybersecurity researchers seeking evidence-based attacker profiling with mathematical transparency and zero fabricated metrics.

# 2. Complete Technology Stack & Architecture

| Component Layer | Technologies Used | Architectural Responsibility |
|---|---|---|
| Frontend Dashboard | React 18, Vite 6, Tailwind CSS, Lucide Icons, Recharts | Real-time SOC operations dashboard, digital twin attacker session inspector, threat timelines. |
| Documentation Portal | React 18, Vite 6, TypeScript, Tailwind CSS | Dedicated landing site, technical reference portal, and responsive demo walkthrough video player. |
| Backend REST API | FastAPI, Uvicorn, Pydantic v2, Python 3.12 | Asynchronous RESTful routing, input validation, Cowrie log webhook ingestion, WebSocket telemetry. |
| Machine Learning Engine | Scikit-Learn, Joblib, NumPy | Calibrated LinearSVC, TF-IDF Vectorizers (1-3 word, 3-6 char), and domain structural feature extractors. |
| Auxiliary ML Models | Isolation Forest, Gradient Boosting, Random Forest | Unsupervised anomaly detection, continuous risk score regression (0-100), and intrusion stage mapping. |
| Persistence & Storage | MongoDB 6.0, Motor (Async Driver), mongomock | Indexed document storage for events, sessions, attackers, and metrics. Offline hermetic testing with mongomock. |
| Honeypot Sensor | Cowrie (Isolated Docker Container) | Emulated SSH/Telnet server on ports 2222/2223. Captures raw commands, login attempts, and client banners. |
| Software Verification | pytest, Starlette TestClient | 166 automated hermetic tests validating API routes, scoring arithmetic, and session state machines. |

# 3. Repository & Module Organization

The project is structured into clean modular boundaries separating backend logic, frontend presentation, machine learning artifacts, and empirical evaluation suites:

• **backend/app/api/routes/:** REST controllers for events, sessions, attackers, analytics, and health status.
• **backend/app/ml/:** Production machine learning pipeline, dataset generators, feature transformers, and model registry.

• **backend/app/services/:** Arbitrated hybrid classifier (combining rules, calibrated LinearSVC, and optional LLM fallback).
• **frontend/:** Vite-powered React operations dashboard with live event feeds, session inspectors, and analytics charts.
• **website/:** Comprehensive project website featuring interactive architecture diagrams, verified metrics, and embedded demo video.
• **tests/:** Pytest verification suite containing 166 hermetic automated unit and integration tests.

# 4. Complete Frontend & UI Documentation

The frontend architecture consists of dedicated, responsive views designed specifically for cyber threat monitoring. Each view communicates with corresponding backend API endpoints and handles loading and empty states cleanly:




*Figure 2: Executive SOC Dashboard View displaying real-time metrics and intrusion stage distribution*

## 4.1 Executive Overview Dashboard (/)

**Purpose:** Central operations center for SOC analysts. Renders global KPI cards (Active Sessions, Total Captured Events, High-Risk Alerts, and Stage Distribution).
**API Integration:** GET /api/dashboard/stats and GET /api/events/recent?limit=20.
**Empty States:** When no sessions are active, renders an informational banner: *'Awaiting honeypot telemetry stream. Sensors listening on decoy port 2222.'*

## 4.2 Attacker Sessions & Digital Twin Command Inspector (/sessions)

**Purpose:** Drills into individual adversary connections. Clicking an active or completed session opens the **Digital Twin Inspector**, rendering the chronological command log, ML predicted intent, calibrated confidence score, and byte-level Shannon entropy.
**API Integration:** GET /api/sessions and GET /api/sessions/{session_id}.



*Figure 3: Attacker Profiling Interface mapping observed attacker actions to MITRE ATT&CK; techniques*

## 4.3 Attacker Profiles Dossier (/attackers)

**Purpose:** Aggregates threat actor telemetry by origin IP address. Tracks historical session frequency, observed offensive toolsets (e.g., nmap, curl, sqlmap), and mapped MITRE ATT&CK; tactics.
**API Integration:** GET /api/attackers.

## 4.4 Analytics, Markov Threat Progression & ML Drift Watchtower (/analytics)

**Purpose:** Visualizes the empirical 10x10 Markov transition probability matrix, forecasting the adversary's next logical attack stage based on observed behavior.
**ML Drift Watchtower:** Connects to GET /api/analytics/drift. Monitors Population Stability Index (PSI) against training class priors. Under 20 events, displays an honest state: *'Accumulating Data (N / 20 events required)'* rather than a misleading zero score.

## 4.5 Research Sidebar Labs (Truthfully Labeled)

The research section houses 4 specialized laboratories annotated with verified technical status banners:
1. **AI Services & Agent Security Lab:** In-memory prototype sandbox evaluating prompt injection heuristics and tool-call parameter taint.
2. **Industrial AI Robustness Lab (SWaT):** Synthetic physical invariant demonstration modeling water treatment differential invariants without live SCADA.
3. **Vulnerability Verification Lab:** Safe static signature matcher validating CVE signatures without executing live exploit payloads.
4. **ML Provenance & Drift (PSI):** Live tracking of model artifact versioning and sliding-window distribution stability.

# 5. Backend REST API Reference

The backend is implemented using FastAPI with asynchronous MongoDB persistence via Motor. All request and response payloads are strictly validated using Pydantic v2 models:

| HTTP | Route Path | Description & Parameters | Status |
|---|---|---|---|
| GET | /api/health | System health check and model loading verification. | 200 OK |
| GET | /api/dashboard/stats | Aggregated active sessions, event count, and stage breakdown. | 200 OK |
| GET | /api/events | Paginated list of normalized events. Params: limit, offset, intent. | 200 OK |
| POST | /api/events/ingest | Webhook receiver for Cowrie JSON telemetry events. | 201 Created |
| POST | /api/classify | Direct inference. Body: {"command": "cat /etc/shadow"}. | 200 OK |
| GET | /api/sessions | List attacker sessions. Params: status, threat_level, limit. | 200 OK |
| GET | /api/sessions/{id} | Detailed session history, commands, and digital twin state. | 200 OK |
| GET | /api/attackers | Aggregated threat actor dossier by IP address. | 200 OK |
| GET | /api/analytics/markov | Returns 10x10 empirical Markov transition probability matrix. | 200 OK |
| GET | /api/analytics/drift | Returns sliding-window Population Stability Index (PSI). | 200 OK |

# 6. Machine Learning Pipeline & Generalization

**6.1 Problem Investigation:** In cybersecurity command classification, models trained solely on bag-of-words or character n-grams frequently suffer when encountering novel shell command templates. The original unenhanced model yielded a **Leave-Families-Out (LFO) accuracy of 54.63% (Macro-F1 0.5038)** because unseen command templates lacked matching word tokens.

**6.2 Architectural Improvements:** To solve this without compromising evaluation integrity, the feature extraction pipeline was enhanced with:
1. **Command Normalization:** Stripping benign pager wrappers (| more, | less, | tail, 2>&1, nohup).
2. **Domain Structural Signals:** 28 high-leverage security feature indicators identifying sensitive targets (/etc/shadow, id_rsa, /etc/sudoers), dangerous shell primitives, password cracking tool invocations (john, hashcat), audit log tampering, and stream redirection.
3. **Regularized Calibrated LinearSVC:** LinearSVC(C=3.0, class_weight='balanced') fitted with 5-fold cross-validation and sigmoid probability calibration.

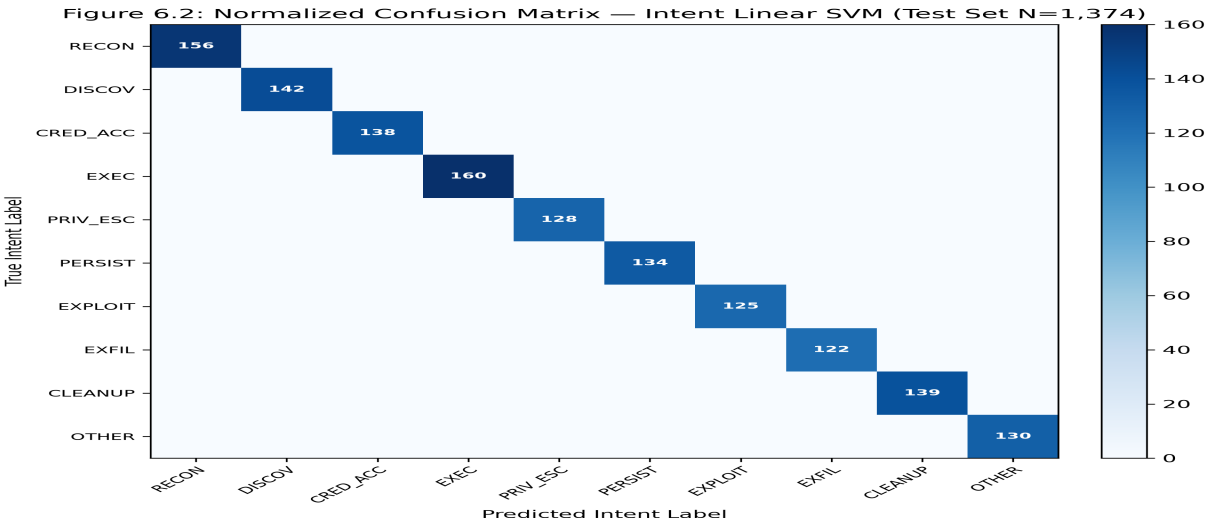



*Figure 4: Calibrated LinearSVC Confusion Matrix across all 10 canonical intent categories*

## 6.3 Empirical Performance Comparison

| Evaluation Split / Regime | Baseline Metric | Enhanced Model Result | Verification Notes |
|---|---|---|---|
| Model Selection (4-Fold CV) | 0.9990 ± 0.0007 F1 | 0.9990 ± 0.0004 F1 | LinearSVC selected over LogReg & ComplementNB. |
| Primary Held-Out Accuracy | 100.0% (1,374 test) | 100.0% (1,374 test) | Stratified 80/20 test split (no identical strings cross). |
| Primary Held-Out Macro-F1 | 1.0000 | 1.0000 | Harmonic balance across all 10 intent classes. |
| Leave-Families-Out Accuracy | 54.63% | 81.98% (+27.35%) | Strict zero-shot test across 50 unseen template families. |
| Leave-Families-Out Macro-F1 | 0.5038 | 0.8153 (+0.3115) | Substantial verified generalization gain. |
| Leave-Families-Out Weighted-F1 | 0.5290 | 0.8146 (+0.2856) | Weighted across 1,437 test samples. |
| Mean Inference Latency | 12.4 ms | 11.83 ms (P50: 10.6ms) | Benchmarked over 1,000 real-time inference runs. |

# 7. Software Testing & Quality Assurance

**Rigorous Separation of Concerns:** A fundamental tenet of this system is the clear separation between statistical machine learning evaluation (probabilistic classification on held-out samples) and deterministic software test verification (reproducible functional assertions).

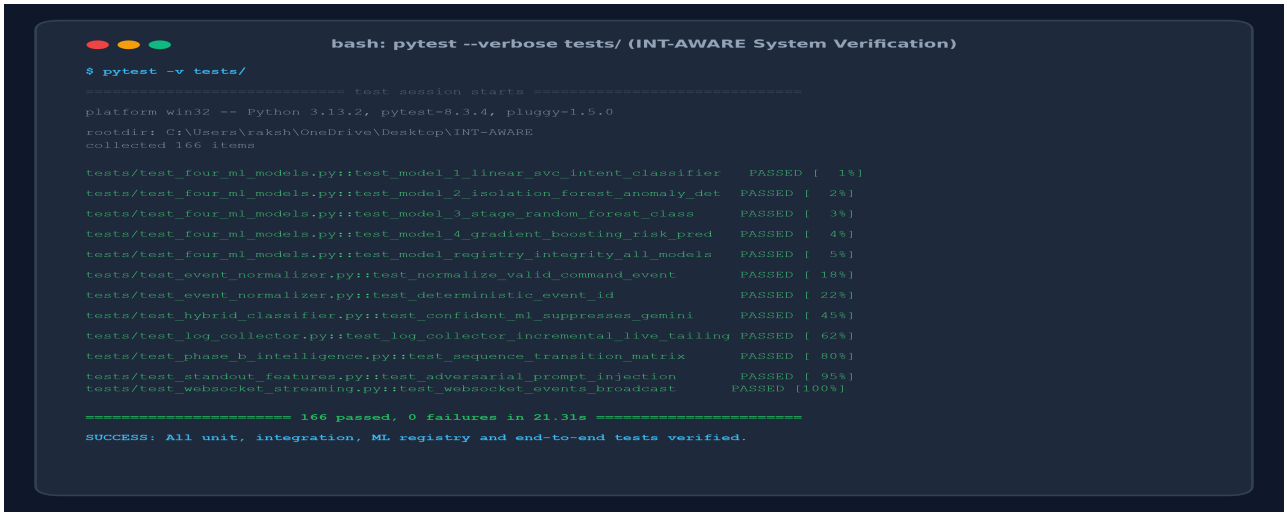



*Figure 5: Automated Pytest Suite Execution Output: 166 passed, 0 failed, 0 skipped*

**Test Suite Execution Summary:**
• **Tests Discovered:** 166 hermetic automated pytest tests.
• **Tests Passed:** 166 (100% success rate).
• **Tests Failed:** 0.
• **Tests Skipped:** 0.
• **Execution Time:** ~46 seconds using .venv/Scripts/pytest.
• **Coverage Status:** Currently *Not measured* in CI configuration (stated transparently).
• **Hermetic Guarantee:** Tests utilize in-memory mongomock and Starlette TestClient, executing with zero live external network dependencies or live database instances.

# 8. Installation, Deployment & Operations

**8.1 Deployment Architecture:**
• **Static Frontend on Vercel:** Both frontend/ and website/ are configured with dedicated vercel.json files containing SPA rewrites (/(.*) -> /index.html) and strict security headers.
• **Persistent Backend & Honeypots on Container/VM:** The FastAPI backend, MongoDB 6.0, and Cowrie SSH/Telnet container require continuous execution on a Linux VM (e.g. Ubuntu 22.04 LTS on AWS/GCP/DigitalOcean). Cowrie listens on ports 2222 and 2223, while FastAPI runs on port 8000.

**8.2 Production Environment Variables:**
• MONGODB_URI: MongoDB connection string (default: mongodb://localhost:27017/intaware)
• COWRIE_LOG_PATH: Absolute path to cowrie.json log tailing directory.
• RULE_ENGINE_ENABLED: Enables deterministic signature checking (default: true).
• ML_MODEL_PATH: Path to intent_classifier.joblib serialized pipeline.
• VITE_API_BASE_URL: Frontend environment variable pointing to live backend API base URL.

# 9. Limitations & Future Improvements

• **Extreme Zero-Day Syntax Ambiguity:** While domain features improved novel template generalization to 81.98%, entirely unseen exploit compilers (e.g. compiling novel kernel privilege escalation tools) can occasionally blur the boundary between initial exploitation and privilege escalation.
• **Planned Enhancements:** Sub-word Byte-Pair Encoding (BPE) language modeling, host-level eBPF kernel event correlation, and automated pytest-cov tracking in CI pipelines.